

5. Envío de datos al servidor con AJAX

Enviar datos al servidor es tan importante como recibirlos. Dependiendo de la situación, podemos usar **GET** o **POST**, o incluso enviar **objetos JSON**.

5.1 Envío de datos usando GET

- Se envían los datos como **parámetros en la URL**.
- Útil para pequeñas cantidades de información.
- Limite de caracteres depende del navegador (aprox. 2000).

Ejemplo práctico: Buscar productos por categoría

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>AJAX GET con parámetros</title>
</head>
<body>
  <h1>Buscar productos</h1>
  <input type="text" id="categoria" placeholder="Categoría">
  <button id="btnBuscar">Buscar</button>
  <ul id="resultados"></ul>

  <script src="script.js"></script>
</body>
</html>
```

JavaScript (**script.js**)

```
document.getElementById("btnBuscar").addEventListener("click",
function() {
  let categoria = document.getElementById("categoria").value;
  let xhr = new XMLHttpRequest();
  xhr.open("GET",
`productos.php?categoria=${encodeURIComponent(categoria)}`, true);

  xhr.onload = function() {
    if (xhr.status === 200) {
      let productos = JSON.parse(xhr.responseText);
      let lista = document.getElementById("resultados");
      lista.innerHTML = "";
      productos.forEach(p => {
        let li = document.createElement("li");
        li.textContent = `${p.nombre} - ${p.precio}`;
        lista.appendChild(li);
      });
    } else {
      alert("Error al buscar productos");
    }
  };
});
```

```
        xhr.send();
    });
```

Archivo `productos.php` (simulado)

```
<?php
$categoria = $_GET['categoria'];
$allProducts = [
    ["nombre"=>"Laptop", "categoria"=>"Electrónica", "precio"=>1200],
    ["nombre"=>"Mouse", "categoria"=>"Electrónica", "precio"=>25],
    ["nombre"=>"Leche", "categoria"=>"Alimentos", "precio"=>2]
];
$result = array_filter($allProducts, fn($p) =>
    strtolower($p["categoria"]) == strtolower($categoria));
echo json_encode(array_values($result));
?>
```

Funcionamiento:

- Los datos se envían en la URL: `productos.php?categoria=Electrónica`.
- El servidor filtra los productos y devuelve un JSON que se muestra dinámicamente.

5.2 Envío de datos usando POST

- Se envían **datos en el cuerpo de la petición**, no en la URL.
- Más seguro y permite enviar **mayor cantidad de información**.

Ejemplo práctico: Enviar formulario de contacto

HTML

```
<form id="formContacto">
    Nombre: <input type="text" name="nombre"><br>
    Email: <input type="email" name="email"><br>
    <button type="submit">Enviar</button>
</form>
<div id="respuesta"></div>
```

JavaScript

```
document.getElementById("formContacto").addEventListener("submit",
function(e) {
    e.preventDefault();
    let formData = new FormData(this);
    let datos = new URLSearchParams(formData.toString());

    let xhr = new XMLHttpRequest();
    xhr.open("POST", "procesar_form.php", true);
    xhr.setRequestHeader("Content-Type", "application/x-www-form-
urlencoded");

    xhr.onload = function() {
```

```

        document.getElementById("respuesta").innerText =
xhr.responseText;
    };

    xhr.send(datos);
});

```

Ventaja:

- Los datos no aparecen en la URL.
- Permite enviar información más sensible.

5.3 Envío de parámetros en formato JSON

- Ideal para enviar **objetos complejos** al servidor.
- Se envían como **string JSON** y se reciben como objeto en el backend.

Ejemplo práctico: Enviar datos de usuario en JSON

```

let xhr = new XMLHttpRequest();
xhr.open("POST", "procesar_json.php", true);
xhr.setRequestHeader("Content-Type", "application/json;charset=UTF-8");

let datos = {
    nombre: "Juan",
    edad: 30,
    email: "juan@mail.com"
};

xhr.onload = function() {
    console.log(xhr.responseText);
};

xhr.send(JSON.stringify(datos));

```

PHP (procesar_json.php)

```

$input = json_decode(file_get_contents('php://input'), true);
echo "Nombre: ".$input['nombre'].", Edad: ".$input['edad'].", Email:
".$input['email'];

```

Ventaja:

- Muy usado en **APIs modernas**.
- Permite enviar **listas y objetos anidados** fácilmente.

5.4 Ejercicios propuestos

1. Crear un formulario que envíe un comentario al servidor y devuelva un mensaje de confirmación usando POST.
2. Modificar el ejemplo GET para que filtre productos por **precio máximo** además de categoría.
3. Crear un objeto JSON con varios usuarios y enviarlo al servidor, que responda con la cantidad de usuarios recibidos.